

Adaptive Resource Management in Software-Defined Networks for IoT Ecosystems

Okwudili Mathew Ugochukwu

Department of Computer Science

Federal University of Lavras

Lavras, Minas Gerais, Brazil

ugochukwu.matthew@estudante.ufla.br

Muhammad Shoaib Ayub,

Pablo Adasme

Department of Electrical Engineering

Universidad de Santiago de Chile

Santiago, Chile

{muhammad.shoaib, pablo.adasme}@usach.cl

Renata Lopes Rosa

Department of Computer Science

Federal University of Lavras

Lavras, Minas Gerais, Brazil

renata.rosa@ufla.br

Demostenes Zegarra Rodriguez

Computer Science Department

Federal University of Lavras

Lavras, Minas Gerais, Brazil

demostenes.zegarra@ufla.br

Muhammad Saadi

Department of Electrical Engineering

University of Central Punjab,

Punjab, Pakistan

muhammad.saadi@ucp.edu.pk

Frederico Gadelha Guimaraes

Department of Computer Science

Federal University of Minas Gerais

Belo Horizonte, Minas Gerais, Brazil

fredericoguimaraes@ufmg.br

Abstract—The proliferation of Internet of Things (IoT) devices has imposed some challenges on traditional network management systems, necessitating diverse approaches for efficient resource allocation. In this paper, we present a framework for adaptive resource management in Software-Defined Networks (SDNs) tailored to IoT ecosystems. Leveraging the inherent flexibility and programmability of SDNs, our methodology integrates reinforcement learning techniques to dynamically allocate network resources based on real-time demands of IoT applications. Our approach introduces a dual-stage Deep Q-Network (DQN) architecture, enhancing stability and accuracy in learning optimal resource allocation policies. This dual-stage DQN, combined with a multi-agent reinforcement learning strategy, maximizes network performance while ensuring fairness and Quality of Service (QoS). The proposed system is evaluated in a simulated IoT environment using the OpenDaylight (ODL) SDN controller. The experimental results demonstrate a significant improvement in key performance metrics, including increased throughput, reduced latency, and decreased energy consumption compared to traditional methods. These enhancements highlight the potential of our adaptive resource management framework in addressing the complex requirements of modern IoT ecosystems.

Index Terms—Internet of Things, network resource allocation and management, Software-Defined Networks.

I. INTRODUCTION

The growth of Internet of Things (IoT) devices has led to an increase in network traffic and complexity, posing significant challenges to traditional network management systems [1], [2]. These systems often lack the flexibility and scalability required to efficiently handle the dynamic and heterogeneous nature of IoT environments. The surge in IoT deployments necessitates innovative approaches to resource management that can adapt to varying demands and optimize network performance [3].

Software-Defined Networking (SDN) has emerged as a revolutionary paradigm that decouples the control plane from the data plane, enabling centralized and programmable network management [4]. This separation allows for more granular control over network resources, making SDN a promising solution for addressing the unique challenges of IoT ecosystems. The programmability of SDN facilitates dynamic resource allocation, which is essential for maintaining the Quality of Service (QoS) in IoT applications that demand real-time processing and low latency [5]. Effective resource management in SDN-based IoT ecosystems involves the dynamic allocation of bandwidth, processing capacity, and memory resources to meet the specific requirements of IoT devices and applications. Traditional resource allocation methods often fall short in these environments due to their static nature and inability to adapt to real-time changes in network conditions. This inadequacy can lead to resource underutilization or over-provisioning, negatively impacting network performance and energy efficiency [6]. To address these challenges, we propose an adaptive resource management framework that leverages reinforcement learning (RL) within the SDN architecture. Specifically, our methodology employs a dual-stage Deep Q-Network (DQN) architecture. This enhancement involves a combination of primary and target networks, facilitating the learning and handling of the temporal correlations in network state observations. The previous work [7] primarily utilized a single DQN model without this sophisticated separation, leading to potential instability and less accurate resource allocation decisions. The proposed framework introduces a multi-agent reinforcement learning approach, enhancing scalability and efficiency by optimizing local resource allocation while coordinating globally. It uses advanced techniques for state

representation with recurrent neural network (RNN) to capture temporal patterns, and improves the reward function to include QoS metrics, energy efficiency, and fairness. The proposed DQN architecture, consisting of primary and target network, in which the primary network is responsible for predicting the Q-values for the current state-action pairs, while the target network is used to provide stable target Q-values for the learning process.

In this system, multiple DQN agents are deployed across various segments of the network, each tasked with optimizing resource allocation within its respective domain.

This paper is organized as follows: Section II reviews related works in the field of SDN-based IoT resource management. Section III details the proposed methodology, including the dual-stage DQN architecture, the multi-agent approach, and the dynamic resource allocation process. Section IV presents the performance evaluation, highlighting the experimental setup and results. Section V discusses the findings and their implications. Finally, Section VI concludes the paper and outlines directions for future research.

II. RELATED WORK

This section presents a review of the current state-of-the-art resource allocation for IoT applications in SDNs. In [8] an intelligent SDN-based Intelligent Decision Support System for IoT (SDN-IDSSIoT) is proposed. This system is composed of two core components: the IDSS conceptual model and the SDN-based architecture, designed to ensure seamless interoperability among diverse smart home devices. The proposed system is evaluated through experiments measuring average throughput and round trip time. However, there are some limitations as the number of devices. In [9] a SDN-based IIoT architecture is proposed with edge computing (EC) outperforming some traditional methods over adaptive computation, effective resource management, and latency in industrial wireless networks (IWNs). However, in [9], only some parameters of the network are measured, such as throughput, and overall delay. Moreover, Javadpour and Wang [10] examined one issue in network virtualization to provide an efficient dynamic resources infrastructure management on Software-Based Networks, in which the method (cTMvSDN) improves resource management based on the combination of Markov-Process and Time Division Multiple Access (TDMA) protocols. A module to initialize the resource mapping is used into it. Thus, the Markov-Pattern and TDMA slicing model is used for predicting the next time gaps, but only the response time and SDN Quality of service are optimized. In [11], a flow-splitting scheme is used to split the incoming flows in the network. Thus, a cost function is used with the aim to route the splittable sub-flows, formulating a min-cost routing problem, known as an integer linear program (ILP) through a detailed greedy heuristic approach. The experimental results show that the proposed scheme presents a higher network throughput by 22% compared to other schemes.

Another proposal [12] improves the RL algorithm to perform a resource allocation with the aim to minimize the

average completion time of jobs and the average number of requested resources. Nevertheless, the experiments are focused only on the convergence performance of the DQN. Some network parameters are not treated, such as the packet loss rate and energy consumption. In [7], another resource allocation method is utilized. However, in it a single DQN model and centralized approach is developed, leading to potential instability and scalability challenges.

Each approach of the related studies addressed only specific challenges and requirements of IoT applications. Our proposed framework incorporates a dual-stage DQN architecture and a multi-agent reinforcement learning approach, resulting in more stable learning and better scalability in large-scale IoT environments.

III. METHODOLOGY

This section provides an overview of the methodology, including the architecture, software tools used, algorithms, and the detailed steps involved in the resource allocation process.

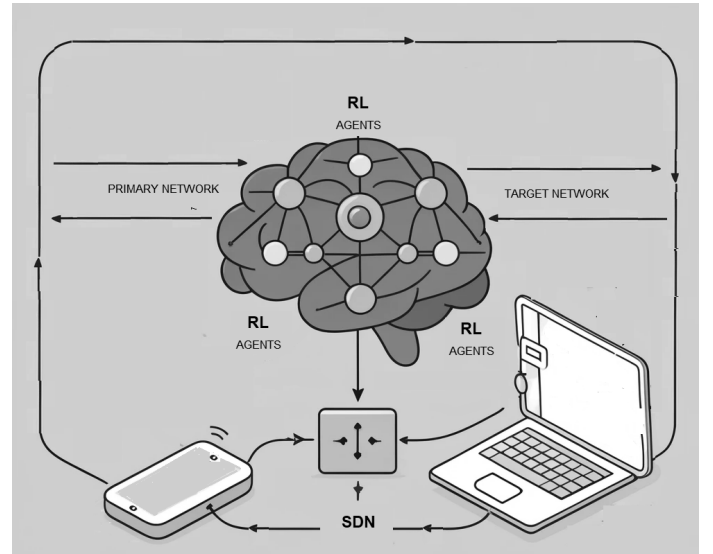


Fig. 1. Proposed adaptive resource management in SDNs for IoT ecosystems.

Fig. 1 provides a high-level overview of the proposed solution architecture, illustrating the interaction between multi-agents for localized resource management, the DRL agent for optimizing performance metrics, and the SDN controller for overall network infrastructure management. Initially, IoT devices continuously generate data and it is sent to the SDN controller, which dynamically manages network resources based on the received information. The RL agent receives data and state information from the SDN controller and applies the RL algorithm to optimize resource allocation within the network. The RL agent sends updated resource allocation strategies back to the SDN controller, which implements these changes in the network. Arrows in the figure indicate the continuous flow of data and control signals between IoT devices, the SDN controller, and the RL agent, ensuring that the network adapts in real-time to changing conditions and

demands. The figure integrates a dual-stage DQN architecture, consisting of primary and target network. The primary network makes predictions based on current network state observations, while the target network is periodically updated with the input of the primary network to enhance learning stability. Additionally, the figure illustrates a multi-agent RL approach, where multiple RL agents operate in different parts of the network. Each RL agent optimizes local resource allocation based on the specific conditions and demands of its network segment. These agents coordinate with each other to achieve global optimization of the network, ensuring that local optimizations contribute to the overall performance and efficiency of the entire network.

A. Architecture Overview

In the architecture of our proposed framework are used the OpenDaylight (ODL) SDN controller for robust network management, a dual-stage Deep Q-Network (DQN) using TensorFlow 2.6 and TF-Agents for stable learning, a multi-agent system for optimizing local and global resource allocation, advanced state representation with RNNs to capture temporal traffic patterns, an enhanced reward function incorporating QoS metrics, energy efficiency, and fairness, simulated IoT devices generating network traffic, and an environment simulator to evaluate performance.

The IEEE 802.11 standard with the Distributed Coordination Function (DCF) is used for the MAC layer, simulating a CSMA/CA environment. This setup ensures fair medium access among multiple devices, significantly influencing throughput and latency.

B. Methodological Clarifications for Reproducibility

1) *Reward Function Calculation:* The reward function in our dual-stage DQN architecture combines several performance metrics, including throughput, latency, energy consumption, and fairness. The reward at each time step t is computed as follows:

$$R_t = \alpha \cdot \frac{T_t}{T_{\max}} + \beta \cdot \left(1 - \frac{L_t}{L_{\max}}\right) + \gamma \cdot \left(1 - \frac{E_t}{E_{\max}}\right) + \delta \cdot F_t \quad (1)$$

where:

- T_t is the throughput at time step t , normalized by the maximum achievable throughput $T_{\max}$.
- L_t is the latency at time step t , normalized by the maximum tolerable latency $L_{\max}$.
- E_t is the energy consumption at time step t , normalized by the maximum possible energy consumption $E_{\max}$.
- F_t is the fairness index at time step t .
- α , β , γ , and δ are the factors for each metric, set to balance their contributions to the overall reward. In our experiments, we set $\alpha = 0.4$, $\beta = 0.3$, $\gamma = 0.2$, and $\delta = 0.1$.

2) *Deep Q-Network (DQN) Architecture:* The DQN used in our framework employs a dual-stage architecture with primary and target networks to enhance learning stability. The primary network is updated at every step, while the target network is updated at regular intervals to match the primary network's input. The hyperparameters used are a learning rate of 0.01, a discount factor of 0.95, a replay memory size of 5000, a batch size of 64, a target network update frequency of 1000 steps, and an exploration factor that decays from 1.0 to 0.1 over 10000 steps. Both the primary and target networks consist of three hidden layers, each with 128 neurons, using ReLU activation functions. The output layer uses a linear activation function to predict Q-values for each possible action.

3) *State Representation:* The state representation includes both static and temporal features of the network. Static features consist of the current traffic load, device priorities, and available resources. Temporal features are captured using a RNN with Long Short-Term Memory (LSTM) cells, configured with 2 LSTM layers, 64 units per layer, and a sequence length of 10 time steps. The combined state representation is a vector concatenating the outputs of the LSTM layers and the static features, providing a comprehensive view of the network's status over time.

4) *Performance Evaluation and Simulation Setup:* To implement and evaluate our proposed framework, we used several software tools: OpenDaylight (ODL) SDN Controller for modular and programmable SDN control, TensorFlow 2.6 for building and training the dual-stage DQN, TF-Agents for reinforcement learning algorithms in TensorFlow, Python for implementing the simulation environment and reinforcement learning agent, and Matplotlib for visualizing experimental results. The simulation setup included 200 IoT devices with varying traffic patterns and resource demands. The key parameters for the simulation are listed in Table I.

A job arrival rate of 30 jobs per second, and ran for 10,000 episodes are configured. Key parameters included an exploration factor of 0.1, replay memory size of 5000, and batch size of 64. The target network update frequency was set to 1000 steps. The simulation lasted 24 hours, high latency sensitivity, and enabled energy consumption monitoring. Traffic patterns varied greatly, initial resource allocation was random, and adaptive resource allocation was enabled.

The IoT devices generated data packets that were processed as computational jobs by the Mobile Edge Computing (MEC) system. The performance of our framework was assessed across multiple episodes to ensure robustness and generalizability. The detailed steps of our proposed method are as follows:

- **IoT Device Requests Resources:** IoT devices generate data packets and request network resources from the SDN controller based on their current requirements.
- **Network State Observation:** The SDN controller continuously monitors the network state, including traffic load, device requirements, and current resource utilization.
- **State Representation:** This involves capturing relevant features such as historical traffic patterns, device pri-

TABLE I
SIMULATION PARAMETERS

Parameter	Value
Number of IoT Devices	200
Computing Resource Levels	250
Job Arrival Rate	30 jobs/second
Number of Episodes	10000
Exploration Factor	0.1
Learning Rate	0.01
Discount Factor	0.95
Replay Memory Size	5000
Batch Size	64
Target Network Update Frequency	1000 steps
Simulation Duration	24 hours
Latency Sensitivity	High
Energy Consumption Monitoring	Enabled
Traffic Pattern Variability	High
Initial Resource Allocation	Random
Adaptive Resource Allocation	Enabled

orities, and resource availability. Advanced state representation techniques, including the use of recurrent neural networks (RNNs), are employed to capture time-dependent patterns in network traffic.

- **Action Selection:** Using an epsilon-greedy strategy, the reinforcement learning agent selects an action that represents a resource allocation decision, which represent specific resource allocation decisions in the network.
- **Reward Calculation:** The reward function evaluates the selected action based on network performance metrics, as throughput, latency, energy consumption, and fairness.
- **Policy Update:** The reinforcement learning agent updates its policy using the observed rewards. This is achieved by training the dual-stage Deep Q-Network (DQN) using experience replay and target network updates to stabilize the learning process.
- **Resource Allocation:** Based on the updated policy, the SDN controller allocates resources to the IoT devices, optimizing network performance. The updated network state is then fed back into the system for continuous learning and adaptation.

The algorithm for our proposal includes multiple agents operating in different parts of the network (reflecting the multi-agent approach), utilizes a dual-stage DQN with separate primary and target networks for stability, and incorporates advanced state representation and reward calculation processes to capture complex network dynamics.

C. Performance Evaluation

The performance evaluation used in our methodology are as follows.

Throughput T is calculated as:

$$T = \frac{TJ}{TT} \quad (2)$$

where TJ is the total data size of all jobs processed, and TT is the total time taken.

Latency is measured as the average time a job spends in the system, including both waiting and processing time.

Algorithm 1: Enhanced DQN-Based Resource Allocation

Data: Initialize replay memory D to capacity N

Data: Initialize primary action-value function Q with random weights

Data: Initialize target action-value function $\hat{Q}$ with $\theta^- = \theta$

Data: Initialize multiple agents for decentralized communic.

```

1 for agent  $i = 1$  to  $n$  do
2   for episode = 1 to  $M$  do
3     Initialize state  $s_{i1}$  from the environment;
4     for  $t = 1$  to  $T$  do
5       if random number  $< \epsilon$  then
6         | Select a random action  $a_{it}$ ;
7       else
8         | Select  $a_{it} = \arg \max_a Q(s_{it}, a; \theta_i)$ ;
9       end
10      Execute action  $a_{it}$  and observe reward  $r_{it}$ 
11      and next state  $s_{i(t+1)}$ ;
12      Store transition  $(s_{it}, a_{it}, r_{it}, s_{i(t+1)})$  in  $D_i$ ;
13      Sample random mini-batch of transitions
14       $(s_{ij}, a_{ij}, r_{ij}, s_{i(j+1)})$  from  $D_i$ ;
15      Set  $y_{ij} =$ 
16      {  $r_{ij}$  if step  $j+1$ 
17      {  $r_{ij} + \gamma \max_{a'} \hat{Q}(s_{i(j+1)}, a'; \theta_i^-)$  otherwise
18      Perform a gradient descent step on
19       $(y_{ij} - Q(s_{ij}, a_{ij}; \theta_i))^2$  with respect to the
20      network parameters;
21      Every  $C$  steps reset  $\hat{Q}_i = Q_i$ ;
22    end
23  end
24 end

```

Data: Combine local policies to form a global policy

Energy consumption E is given by:

$$E = P_{\text{MEC}} \times TPT \quad (3)$$

where P_{MEC} is the power consumption of the MEC system, and TPT is the total processing time.

Fairness F is calculated using the Jain's Fairness Index:

$$F = \frac{(\sum_{i=1}^n x_i)^2}{n \sum_{i=1}^n x_i^2} \quad (4)$$

where x_i is the resource allocated to the i -th device and n is the number of devices.

Jain's index is chosen for its widespread acceptance and effectiveness in evaluating resource distribution fairness in IoT networks, providing a comprehensive measure compared to indices like the Gini coefficient and Theil index.

D. Use Case Scenario

To evaluate the effectiveness of our proposed framework, we conducted experiments in a simulated IoT environment. The simulation involves 200 IoT devices distributed within

the network's service region. Each device generates data packets that are processed as computational jobs by the MEC system. The resource allocation process is evaluated over multiple episodes to measure key performance metrics such as throughput, latency, energy consumption, and fairness.

IV. RESULTS AND DISCUSSIONS

This section presents an analysis of the results obtained from our proposed framework for adaptive resource management in SDNs tailored to IoT ecosystems. We evaluate the performance of our framework in terms of throughput, latency, energy consumption, and fairness. The results are compared with traditional methods and recent advanced algorithms to highlight the effectiveness of our approach.

The simulation environment was designed to capture the dynamic and unpredictable behavior typical of IoT networks. Several factors contribute to the stochastic nature of the simulation:

- **IoT Device Behavior:** Each IoT device generates data packets at random intervals, following a Poisson distribution with an average arrival rate of 30 jobs per second. This randomness in job generation simulates the variable nature of IoT traffic.
- **Network Conditions:** The network conditions, such as latency and bandwidth availability, vary over time based on real-time demands and traffic patterns, which are also randomly generated to reflect realistic scenarios.
- **Reinforcement Learning Exploration:** The reinforcement learning agent uses an epsilon-greedy strategy for action selection, introducing randomness in the decision-making process. This helps the agent explore different resource allocation strategies and avoid local optima.

To ensure the robustness and reliability of our results, each simulation used different random seeds. The results presented in the following sections are the averages of 10,000 episodes, and the standard deviations are included to provide a measure of variability.

A. Throughput Analysis

Our proposed DQN-based resource allocation framework shows significant improvements in throughput compared to traditional methods. The throughput values reported are the averages over multiple simulation runs. In the figures, our proposed method is referred to as DQN2, while the others, DQN [12] and DQN1 [7], are methods from related works.

Figure 2 shows the throughput achieved by different methods. Our proposed framework, DQN2, achieves a throughput of 12.15 Gbps on average, with a standard deviation of 0.3 Gbps. This is significantly higher than advanced methods like DQN (10 Gbps), cTMvSDN (10.33 Gbps), and QoS-aware (10.98 Gbps). The integration of dual-stage DQN and multi-agent reinforcement learning contributed to this improvement for the reason the agents coordinate with each other to achieve global network optimization, ensuring that local improvements contribute to the overall network performance.

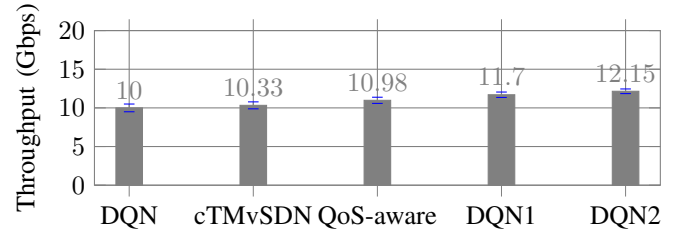


Fig. 2. Throughput comparison of different methods. The bars represent the mean throughput, and the error bars represent the standard deviation.

The throughput of 12.15 Gbps in our proposed framework, DQN2, simulates a high-density IoT environment with mixed traffic conditions, demonstrating its applicability to both low and high-data-rate IoT devices and its robustness in various IoT settings.

B. Latency Analysis

The latency values reported below are the averages over multiple simulation runs. As shown in Figure 3, our proposed framework, DQN2, achieves the lowest latency of 13.8 ms on average, with a standard deviation of 0.6 ms. This significant improvement over other methods, including a 31% reduction compared to the DQN (20 ms), underscores the effectiveness of our dual-stage in which the RL agents can make more informed decisions about resource allocation. This precise understanding helps to avoid unnecessary delays and optimizes the flow of data through the network.

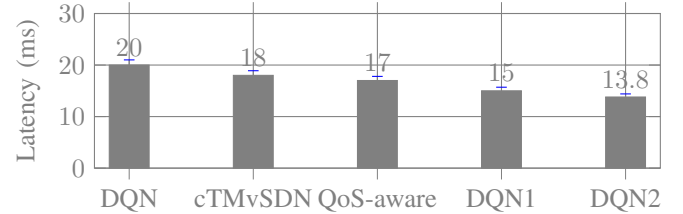


Fig. 3. Latency comparison of different methods. The bars represent the mean throughput, and the error bars represent the standard deviation.

C. Energy Consumption Analysis

The energy consumption values reported are the averages over multiple simulation runs. Figure 4 illustrates the energy consumption of different methods. Our proposed framework, DQN2, achieves the lowest energy consumption of 128 mJ on average, with a standard deviation of 6.4 mJ. This reduction, achieved through advanced state representation and a comprehensive reward function, demonstrates the efficiency of our resource management approach in reducing the energy footprint of IoT networks.

The reward function incentivizes the RL agents to make decisions that optimize the use of resources while minimizing energy consumption. By balancing performance and energy efficiency, the framework ensures that energy is used only when necessary.

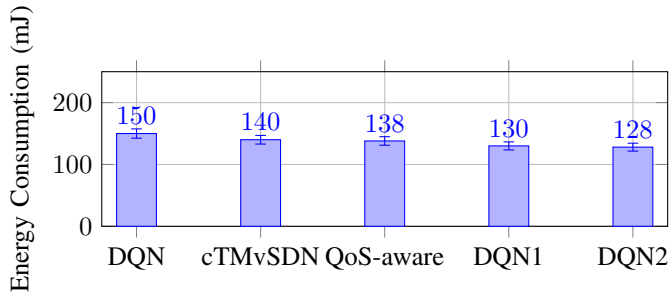


Fig. 4. Energy consumption comparison of different methods. The bars represent the mean throughput, and the error bars represent the standard deviation.

D. Fairness Analysis

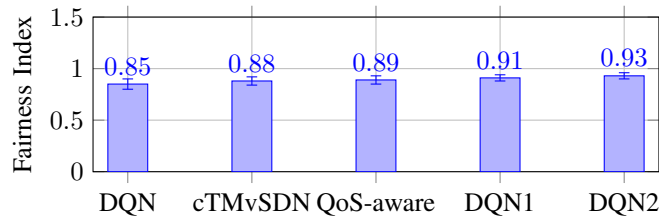


Fig. 5. Fairness comparison of different methods. The bars represent the mean fairness index and the error bars represent the standard deviation.

As depicted in Figure 5, our proposed framework achieves the highest fairness index of 0.93 on average, with a standard deviation of 0.03. This indicates a more equitable distribution of resources among IoT devices compared to DQN (0.85), cTMvSDN (0.88), QoS-aware (0.89), and DQN1 (0.91). The multi-agent approach and enhanced reward function contribute to this improvement by ensuring fairness in resource allocation. The fairness values reported are the averages over multiple simulation runs.

V. CONCLUSION

The experimental results validate our proposed dual-stage DQN-based resource allocation framework in enhancing network performance across key metrics. By integrating reinforcement learning into the SDN architecture, our framework enables continuous adaptation and optimization of resource allocation strategies based on real-time network conditions. This adaptability is essential for maintaining high performance in ever-changing IoT environments. The enhancements introduced in our framework, such as the dual-stage DQN architecture, multi-agent reinforcement learning approach, and advanced state representation with RNNs, contribute to the observed performance gains. These innovations ensure more stable learning, better handling of temporal correlations, and more holistic optimization of network performance. DQN2 outperforms DQN [12] and DQN1 [7] due to the integration of a dual-stage DQN architecture, which enhances learning stability and handles temporal correlations in network state observations more effectively. Furthermore, the use of advanced

state representation with RNNs allows for better capturing of temporal traffic patterns, and the multi-agent reinforcement learning approach optimizes local and global resource allocation. The results highlight the potential of the proposed framework to be extended to other network scenarios, including more complex and large-scale IoT deployments. Future work will focus on exploring these extensions and further refining the reinforcement learning algorithms to achieve even better performance. Future research will explore the application of our framework to diverse IoT scenarios and investigate the integration of other machine learning techniques to enhance its robustness and scalability. Additionally the proposed method will be compared to other multi-agent deep reinforcement learning methods.

REFERENCES

- [1] M. O. Akinsanya, C. C. Ekechi, and C. D. Okeke, "Security paradigms for iot in telecom networks: conceptual challenges and solution pathways," *Engineering Science & Technology Journal*, vol. 5, no. 4, pp. 1431–1451, 2024.
- [2] R. L. Rosa, D. Z. Rodriguez, and G. Bressan, "Sentimeter-br: A social web analysis tool to discover consumers' sentiment," in *2013 IEEE 14th international conference on mobile data management*, vol. 2. IEEE, 2013, pp. 122–124.
- [3] R. Jeyaraj, A. Balasubramaniam, A. K. MA, N. Guizani, and A. Paul, "Resource management in cloud and cloud-influenced technologies for internet of things applications," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–37, 2023.
- [4] A. Narwaria and A. P. Mazumdar, "Software-defined wireless sensor network: A comprehensive survey," *Journal of Network and Computer Applications*, p. 103636, 2023.
- [5] N. N. Josbert, M. Wei, W. Ping, and A. Rafiq, "A look into smart factory for industrial iot driven by sdn technology: A comprehensive survey of taxonomy, architectures, issues and future research orientations," *Journal of King Saud University-Computer and Information Sciences*, p. 102069, 2024.
- [6] M. A. Zormati and H. Lakhlef, "An overview of machine learning-enabled network softwarization for the internet of things," in *2023 International Conference on Software, Telecommunications and Computer Networks (SoftCOM)*. IEEE, 2023, pp. 1–6.
- [7] A. L. de Sousa, O. D. OKey, R. L. Rosa, M. Saadi, and D. Z. Rodriguez, "A novel resource allocation in software-defined networks for iot application," in *2023 International Conference on Software, Telecommunications and Computer Networks (SoftCOM)*. IEEE, 2023, pp. 1–5.
- [8] K. N. Qureshi, A. Alhudhaif, M. Azahar, I. T. Javed, and G. Jeon, "A software-defined network-based intelligent decision support system for the internet of things networks," *Wireless Personal Communications*, vol. 126, no. 4, pp. 2825–2839, 2022.
- [9] S. Chandramohan, M. Senthikumar, and M. Sivakumar, "Adaptive computing optimization for industrial IoT using sdn with edge computing," in *International Conference on Computing Methodologies and Communication*, 2022, pp. 360–365.
- [10] A. Javadpour and G. Wang, "cTMvSDN: improving resource management using combination of Markov-process and TDMA in software-defined networking," *The Journal of Supercomputing*, pp. 1–23, 2022.
- [11] P. Kamboj, S. Pal, S. Bera, and S. Misra, "QoS-aware multipath routing in software-defined networks," *IEEE Transactions on Network Science and Engineering*, vol. 10, no. 2, pp. 723–732, 2023.
- [12] X. Xiong, K. Zheng, L. Lei, and H. Lu, "Resource allocation based on deep reinforcement learning in IoT edge computing," *IEEE Journal on Selected Areas in Communications*, vol. PP, pp. 1–1, 04 2020.